UNIVERSITY OF SCIENCE - VNUHCM

FACULTY OF INFORMATION TECHNOLOGY

SOFTWARE TESTING

HW6 - PERFORMANCE TESTING

Software Testing Project Report

Authors:
Trần Thị Cát Tường
(22127444)
ttctuong22@clc.fitus.edu.vn

Supervisors:
Teacher Trần Duy Hoàng
Teacher Hồ Tuấn Thanh
Teacher Trương Phước Lộc

August 11, 2025

Table of Contents

1 Group Information

Group ID: 07

Member Name	Student ID	Assigned Features	Status
Cao Uyển Nhi	22127310	Homepage - Category - Product	Done
Lưu Thanh Thuý	22127410	Register	Done
Nguyễn Phước Minh Trí	22127424	Homepage - Sort - Search	Done
Võ Lê Việt Tú	22127435	Sign In - Invoice	Done
Trần Thị Cát Tường	22127444	Homepage - Contact	Done

2 Performance Testing Summary

2.1 Feature Description

Feature Name: Contact Form Submission

Business Flow:

1. **Access Home Page**

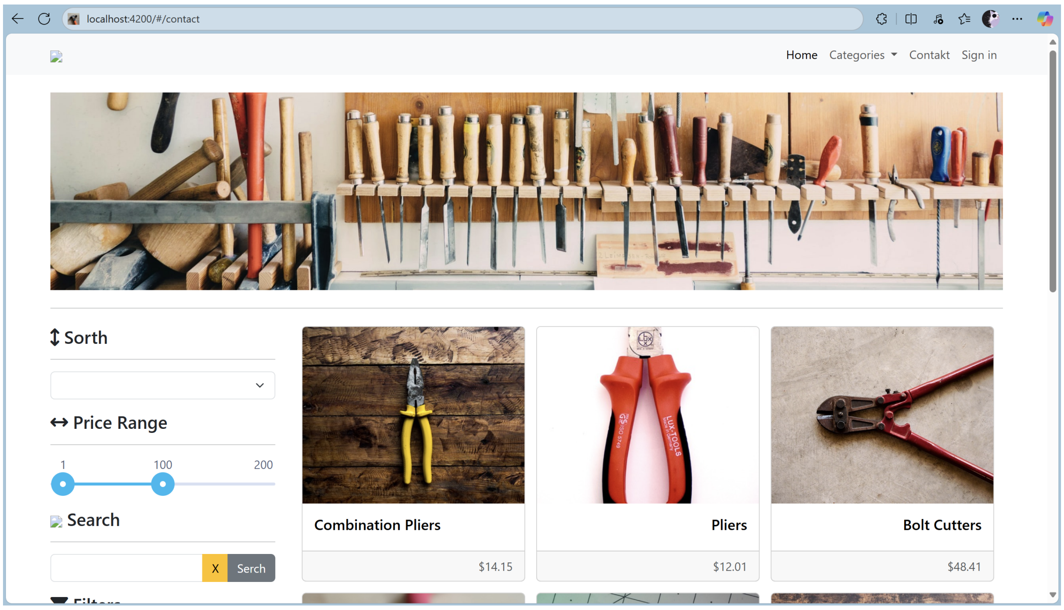

Figure 1: Homepage

- Navigate to:

  `GET http://localhost:4200/`

  or

  `GET http://localhost:4200/#/`

- The following **fetch/XHR** requests are sent to the backend:
  - `GET http://localhost:8091/users/me` *(not required for guests since no login is performed)*
  - `GET http://localhost:8091/products?between=price,1&page=1`
  - `GET http://localhost:8091/brands`
  - `GET http://localhost:8091/categories/tree`

2. **Access Contact Page**

- Navigate to:

  `http://localhost:4200/#/contact`

- The following **fetch/XHR** requests are sent to the backend:
  - `GET http://localhost:8091/users/me` *(not required for guests)*

3. **Submit Contact Form**

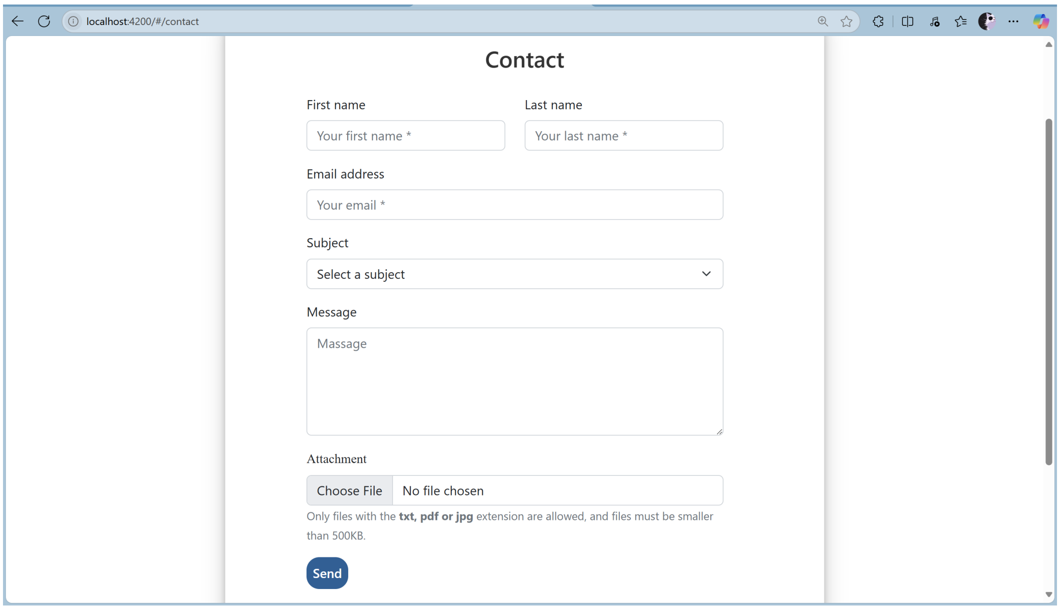

Figure 2: Contact function interface for guest

- **UI fields:**
  - First name
  - Last name
  - Email address
  - Subject (dropdown options):
    - Customer service
    - Webmaster
    - Return
    - Error 101: Subject not found
    - Payments
    - Warranty
    - Status of my order
    - Error 202: Translation error
  - Message (text area)
  - File attachment (optional)
- **Requests sent:**

```
POST http://localhost:8091/messages
{
  "name": "Joe Doe",
  "email": "customer@practicesoftwaretesting.com",
  "subject": "customer-service",
  "message": "yeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeee
  eeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeee",
  "status": "NEW"
}
```

Note: Currently, file attachment feature returns an error: *"only empty files are accepted"*.

Business Priority: High – The contact form is a direct customer communication channel and a critical part of customer support operations.

2.2 Test Environment

Hardware Configuration:

- **CPU:** 12th Gen Intel® Core™ i5-12500H @ 2.50GHz (12 cores, 16 threads)
- **RAM:** 16 GB DDR4
- **Storage:** 512 GB NVMe SSD (System drive: C:)
- **Operating System:** Windows 11 Home Single Language, Version 23H2 (OS Build 22631.5624)

Software & Tools:

- Apache JMeter 5.6.3
- JMeter Plugins:
  - Ultimate Thread Group

System Under Test (SUT) Deployment:

- **Application:** The Toolshop – Sprint 5 with bugs version
- **Backend:** Laravel API
- **Frontend:** Angular application served at `http://localhost:4200/`
- **API Base URL:** `http://localhost:8091`

2.3 Common Configuration (applied to all test scripts)

Config Elements:

- **CSV Data Set Config** – Reads test data from an external CSV file.
- **HTTP Request Defaults** – Sets the default server name, port, and protocol for HTTP requests.
- **HTTP Header Manager** – Defines default HTTP headers (e.g., Content-Type, Accept).
- **HTTP Cookie Manager** – Handles cookies to maintain session state during test execution.
- **Uniform Random Timer** – Adds a random delay between requests to simulate more realistic user behavior.

Assertions:

- **Response Assertion** – Validates that the HTTP response code is either 200 or 201.

Listeners:

- **Summary Report** – Displays aggregate metrics (samples, average time, error %, throughput, etc.).
- **Aggregate Graph** – Visualizes performance metrics in chart form.

- **View Results Tree** – Displays detailed request/response information for debugging.

2.4 AI-Generated Test Data

To create realistic and comprehensive input for the Contact Form Submission scenario, an AI tool (ChatGPT) was prompted to generate a CSV dataset following the **Equivalence Partitioning (EP)** criteria below:

Field	Valid Partition	Invalid Partition
First Name	Non-empty, $\leq$ 120 characters	Empty, $>$ 120 characters
Last Name	Non-empty, $\leq$ 120 characters	Empty, $>$ 120 characters
Email	Valid email format (e.g. `a@b.com`)	Missing `@`, missing domain, empty, malformed format, $>$ 120 characters
Subject	A valid selected option (not default/empty)	Not selected (empty/default value)
Message	Non-empty, $\geq$ 50 and $\leq$ 250 characters	Empty, $<$ 50 characters, $>$ 250 characters
Attachment	`.txt`, `.pdf`, `.jpg`, file size $\leq$ 500KB	Wrong type (e.g. `.docx`, `.exe`), size $>$ 500KB

Prompt Transparency: The AI was explicitly instructed with:

```
Generate multiple CSV rows in the format firstName,lastName,email,
subject_key,message,expect.
All rows must strictly comply with the valid partition rules from the
provided
Equivalence Partitioning (EP) table. Ensure that each field value meets
its corresponding
validity criteria, and include boundary conditions for fields with length
constraints.
The expect column should be set to valid for all rows."

[Equivalence Partitioning Table]
```

Added Value: By using AI, we rapidly produced:

- Large, varied datasets without repetitive manual entry.
- Edge cases and boundary conditions that are often overlooked in manual preparation.
- Consistent formatting suitable for direct import into JMeter's CSV Data Set Config.

2.5 Specific Test Configurations

2.5.1 Load Testing Configuration

Thread Group:

- **Type:** Standard Thread Group
- **Load Profile:**
  - **Number of Threads (users):** 40
  - **Ramp-up Period:** 30 seconds
  - **Loop Count:** Infinite
  - **Duration:** 180 seconds
- **Execution Logic:** All users perform the Contact Form Submission scenario with parameterized data from the CSV Data Set Config.

Purpose:

- Simulate the expected average load in production.
- Measure baseline response time, throughput, and error rate under normal operating conditions.

Observations:

- All requests completed successfully with 0% error rate.
- The average response time across all requests was ~3.56 seconds.
- The slowest request was POST /messages with a maximum of 10.23 seconds and an average of ~3.95 seconds.
- Throughput remained stable at ~8.9 requests/sec overall.
- Minimum recorded latency was 142 ms (GET /products), showing that under light instantaneous load, the server can respond quickly.

View Results Tree:

- All samplers returned HTTP 200 OK.
- Response payloads matched expected results from the Contact Form Submission flow.

Conclusion:

Under normal load, the system handled requests consistently without errors. However, average latencies above 3 seconds for all endpoints, especially POST operations, suggest potential optimization opportunities in backend processing.

2.5.2 Stress Testing Configuration

Thread Group:

- **Type:** Ultimate Thread Group
- **Load Pattern:**

Start Threads	Initial Delay (s)	Startup Time (s)	Hold Load For (s)	Shutdown Time (s)
20	0	10	60	10
200	70	5	30	5
20	110	10	60	10
0	190	1	1	1

Execution Logic: Gradually increase the number of concurrent users in stepped increments until the system reaches or exceeds its maximum sustainable capacity.

Purpose:

- Identify the maximum load threshold before system degradation.
- Detect potential bottlenecks in processing requests under high load.

Observations:

- No errors were recorded despite the high and gradually increasing load.
- Average response times are significantly higher than in Load Testing:
- From ~3.5s in load testing to ~20s under stress conditions.
- The 90% Line for all requests exceeded 34 seconds, with the highest reaching ~48 seconds for GET /brands.
- Throughput increased to ~25.6 requests/sec due to the higher number of concurrent users, but at the cost of much slower response times.
- POST /messages still had the slowest throughput (4.3 req/sec), indicating that write operations are more sensitive to load increases.

View Results Tree:

- All requests returned HTTP 200 OK with valid response payloads.
- No functional errors occurred, but high response times indicate severe performance degradation under heavy load.

Conclusion:

The system maintained functional correctness under extreme load but exhibited severe latency increases, which would likely impact user experience. Performance bottlenecks are most evident in GET /brands and POST /messages, suggesting the need for backend query optimization and possibly caching strategies.

2.5.3 Spike Testing Configuration

Thread Group:

- **Type:** Ultimate Thread Group
- **Load Pattern:**

Start Threads	Initial Delay (s)	Startup Time (s)	Hold Load For (s)	Shutdown Time (s)
50	0	10	90	15
200	115	10	90	15
400	230	10	90	15
600	345	10	90	15
800	460	10	90	15
1000	575	10	90	15
1200	690	10	90	15
50	805	15	120	15

Execution Logic: Apply sudden and substantial increases in concurrent users to simulate traffic spikes, then drop load to baseline to assess recovery.

Purpose:

- Evaluate the system's ability to handle sudden surges in traffic.
- Assess how quickly and effectively the system recovers after peak loads.

Observations:

- All requests completed successfully (0% error rate) despite rapid load surges and drops.
- Average latencies were higher than in load testing but significantly lower than in stress testing:
- ~4–5 seconds average compared to ~20 seconds in stress.
- The 90% Line values indicate that during peak spikes, some requests reached latencies of 13–15 seconds.
- Throughput peaked at ~11 requests/sec overall, reflecting alternating high-load and recovery phases.
- POST /messages again had the lowest throughput, confirming it as the most performance-sensitive operation.

View Results Tree:

- All samplers returned HTTP 200 OK with expected response bodies.
- No failures occurred during spike peaks or recovery periods.

Conclusion:

The system handled sudden surges in concurrent users without errors, showing good stability under spike conditions. However, latency during peak spikes is still high enough to potentially degrade user experience, particularly for POST requests.

2.6 Youtube Video Link

- Load Testing
- Stress Testing
- Spike Testing

3 Self-Evaluation

Criteria	Description	Max Points	Self Assessment
Load testing	Report: 1.0 TestCases, BugReport: 0.5 Script, Data: 0.5 Video: 1.0	3.0	3.0
Stress testing	Report: 1.0 TestCases, BugReport: 0.5 Script, Data: 0.5 Video: 1.0	3.0	3.0
Spike testing	Report: 1.0 TestCases, BugReport: 0.5 Script, Data: 0.5 Video: 1.0	3.0	3.0
Use of AI Tools	Prompt transparency, critical validation, added value	1.0	1.0
Total		**10.0**	**10.0**